

Relatório Técnico: Trabalho Prático 1

Processamento de Linguagem Natural em Engenharia Biomédica

João Silva (PG61212), Miguel Sousa (PG59904)

Abril de 2026

1 Introdução

Este projeto insere-se no âmbito do Trabalho Prático 1 da unidade de Processamento de Linguagem Natural (PLN). O objetivo é converter o dicionário médico *medicina.pdf* e outros ficheiros .pdf num formato estruturado JSON. Este processo envolve técnicas de limpeza de texto, análise de padrões através de expressões regulares (Regex) e modelação de dados.

2 Descrição Geral do Problema

Os diferentes documentos fornecidos apresentam-se em formatos não estruturados, dificultando a sua utilização direta em aplicações computacionais. De forma geral, os principais problemas identificados foram:

- Presença de ruído (cabecinhos, números de página);
- Quebras de linha inconsistentes;
- Estruturas heterogéneas entre documentos;
- Falta de uniformização na representação da informação.

Cada documento exigiu uma abordagem específica, devido às suas particularidades.

3 Arquitetura do Sistema

Para dar resposta à necessidade de processar múltiplos ficheiros PDF com estruturas totalmente distintas, adotou-se uma arquitetura de software modular e escalável. Esta abordagem evitou o desenvolvimento de um script monolítico, facilitando a manutenção do código, o isolamento de erros e a eventual integração futura de novos dicionários.

A estrutura de diretórios do projeto foi organizada da seguinte forma, refletindo um fluxo de trabalho claro (desde os dados em bruto até ao *output* final processado):

```
TP1/
|-- Dicionarios/      # Output (ficheiros .json criados)
|-- Parsers/         # Lógica para extrair (ficheiros .py)
|-- Pdfs/            # Ficheiros originais (ficheiros .pdf)
|-- Textos/          # Output intermédio criado através do pdftotext
|
|-- main2.py          # Script principal
|-- pdftotext.exe     # Utilitário para a extração de texto
```

3.1 Descrição geral dos componentes

O funcionamento deste sistema de Processamento de Linguagem Natural baseia-se na interação entre as diferentes pastas e ficheiros apresentados:

- **Pdfs e Textos:** A pasta `Pdfs` armazena os documentos originais inalterados, garantindo a preservação da fonte. O utilitário `pdftotext.exe` atua sobre estes ficheiros, gerando o texto em formato *plain text* que é armazenado na pasta `Textos`. Esta separação permite diagnosticar facilmente se um erro de extração tem origem no conversor de PDF ou no algoritmo de *parsing*.
- **Parsers:** Esta pasta contém os algoritmos centrais do projeto. Foi criado um script Python independente para cada documento (e.g., `parser_covid.py`, `parser_medicina.py`), onde cada script contém as expressões regulares e heurísticas dedicadas exclusivamente ao formato do seu respetivo documento. Para além disso, foi também incluído nesta pasta o script `merger_json2.py`, responsável pela convergência e normalização dos dados.
- **main2.py:** Atua como o ponto de entrada do sistema e não possui lógica de extração; a sua função é importar os módulos da pasta `Parsers`

e executá-los sequencialmente. Isto permite ativar ou desativar o processamento de ficheiros específicos de forma simples durante as fases de teste.

- **Dicionários:** Armazenam os ficheiros JSON individuais gerados por cada parser, bem como o ficheiro `dicionario_global.json`, que agrega toda a terminologia extraída, cumprindo assim o objetivo final do trabalho prático.

3.2 Pasta PDF's

3.2.1 *medicina.pdf*

O documento de medicina revelou-se um dos mais estruturados entre os analisados, apresentando entradas organizadas por identificadores numéricos. No entanto, apesar desta aparente organização, foram identificados alguns desafios relevantes relacionados com a formatação do texto e a consistência das traduções.

Em particular, verificaram-se inconsistências nas quebras de linha e na forma como as diferentes línguas estavam distribuídas ao longo das entradas, o que dificultou o processamento direto.

- Estrutura relativamente organizada por IDs;
- Presença de múltiplas línguas por entrada;
- Quebras de linha dentro das traduções.

3.2.2 *WIPOPearl_COVID19_Glossary.pdf*

O documento relativo ao glossário de COVID-19 apresentou um nível de complexidade superior, devido à sua estrutura apenas parcialmente organizada. Ao contrário do documento de medicina, este não seguia um padrão rígido, o que obrigou a uma abordagem mais flexível e robusta.

Além disso, a presença de ruído textual e inconsistências nos delimitadores entre entradas tornou o processo de identificação de termos mais desafiante.

- Estrutura semi-organizada;
- Inconsistência nos delimitadores entre entradas;
- Elevado nível de ruído textual.

3.2.3 *glossario_neologismos_saude.pdf*

O documento de neologismos destacou-se pela sua natureza menos formal e pela ausência de uma estrutura claramente definida. Este fator dificultou significativamente a identificação automática dos elementos principais, como o termo e a respetiva definição.

Foi necessário recorrer a heurísticas baseadas em padrões linguísticos e contexto textual, tornando o processo mais sensível a variações no conteúdo.

- Estrutura pouco rígida;
- Dificuldade em identificar claramente termos e definições;
- Dependência de padrões linguísticos.

3.2.4 *glossario_ministerio_saude.pdf*

O documento proveniente do Ministério da Saúde apresenta uma organização distinta, baseada em categorias temáticas. Esta estrutura hierárquica introduz um nível adicional de complexidade, uma vez que os termos não são apenas sequenciais, mas agrupados segundo contextos específicos.

Assim, foi necessário preservar a relação entre categorias e termos durante o processamento, garantindo que a informação semântica não fosse perdida.

- Organização por categorias;
- Estrutura hierárquica;
- Necessidade de manter contexto entre secções.

3.2.5 *Glossário de Enfermagem.pdf*

O documento de enfermagem apresenta uma estrutura baseada em pares termo-definição, mas com elevada inconsistência na formatação. As quebras de linha irregulares e a ausência de delimitadores claros entre entradas dificultaram a segmentação automática.

Este tipo de documento exigiu uma abordagem mais sequencial, baseada na análise da posição do texto e na identificação de padrões implícitos.

- Estrutura baseada em pares termo-definição;
- Quebras de linha irregulares;
- Formatação inconsistente.

4 Extração de Informação

Após a análise dos diferentes documentos, foi necessário definir estratégias específicas para a extração da informação, tendo em conta as particularidades estruturais de cada fonte. Devido à heterogeneidade dos formatos, optou-se por adotar uma abordagem única, sendo antes desenvolvido um conjunto de parsers especializados, cada um adaptado ao tipo de documento em questão.

Antes de se proceder à manipulação dos dados, a etapa primordial consistiu na conversão dos documentos originais, em formato PDF, para ficheiros de texto simples (.txt). Para este efeito, recorreu-se à ferramenta de linha de comandos `pdftotext`, utilizando parâmetros específicos (como o `-raw`) para preservar, na medida do possível, o fluxo e o layout original das linhas de texto.

Uma vez obtido o texto bruto, o processo de extração seguiu uma sequência de etapas comuns. Inicialmente, procedeu-se à limpeza e normalização do texto, removendo elementos irrelevantes como cabeçalhos, números de página e quebras de linha inconsistentes. De seguida, foram identificados padrões estruturais recorrentes, que permitiram segmentar o texto em unidades lógicas correspondentes a termos ou entradas do dicionário. Posteriormente, foi realizada a extração da informação relevante, como termos, definições ou traduções, dependendo do documento. Por fim, os dados foram organizados em estruturas adequadas para posterior conversão num formato unificado JSON.

As expressões regulares desempenharam um papel central neste processo, permitindo identificar padrões mesmo em documentos com formatação inconsistente. No entanto, em vários casos, foi necessário complementar esta abordagem com heurísticas adicionais, de forma a aumentar a robustez do sistema e garantir uma extração mais fiável.

Nos pontos seguintes descreve-se, de forma detalhada, a metodologia adotada para cada documento.

De forma geral, o processo integral de extração seguiu a seguinte sequência de etapas:

- Conversão dos documentos PDF para texto simples (.txt) com a ferramenta `pdftotext`;
- Limpeza e normalização do texto extraído;
- Identificação de padrões estruturais;
- Segmentação em unidades lógicas (entradas ou termos);
- Extração da informação relevante;

- Construção de estruturas de dados organizadas para conversão final.

Nos pontos seguintes descreve-se, de forma detalhada, a metodologia adotada para cada documento.

4.1 Extração do texto simples

A primeira etapa de processamento consistiu na conversão dos documentos originais em formato PDF para ficheiros de texto simples (`.txt`). Uma condição necessária para facilitar a manipulação programática dos dados. Para garantir que a estrutura lógica das definições e termos não fosse corrompida, utilizou-se a ferramenta de linha de comandos `pdftotext` (parte do pacote `XpdfReader`).

O parâmetro fundamental utilizado em todos os comandos foi o `-raw`. Este modo de extração ignora a disposição física das colunas e caixas de texto na página, extraíndo o conteúdo na ordem em que aparece no fluxo interno do PDF, o que se revelou essencial para manter a integridade de termos e definições que se estendiam por múltiplas colunas ou páginas.

Ao longo do trabalho, foram executados os seguintes comandos específicos para cada fonte de dados:

1. **Glossário de Neologismos:** Para este documento, foi necessário ignorar as páginas iniciais de introdução e índice, iniciando a extração apenas na página 15, onde começavam os conceitos.

```
.\pdftotext.exe -f 15 -raw neologismos.pdf Textos/neologismos.txt
```

2. **WIPO Pearl COVID-19 Glossary:** Devido à densidade de informação e presença de múltiplos idiomas, a extração integral foi realizada mantendo o fluxo bruto do texto.

```
.\pdftotext.exe -raw WIPOPearl_COVID-19_Glossary.pdf Textos/covid19.txt
```

3. **Glossário de Medicina (Obrigatório):** O ficheiro obrigatório exigido pelo enunciado foi convertido de forma a ser processado pelo parser especializado.

```
.\pdftotext.exe -raw medicina.pdf Textos/medicina.txt
```

4. **Glossário do Ministério da Saúde:** Para o documento organizado por categorias, utilizou-se o mesmo método de preservação do fluxo de texto.

```
.\pdftotext.exe -raw glossario_ms.pdf Textos/texto_ms.txt
```

5. **Glossário de Enfermagem:** Para o documento organizado em termos e descrições, utilizou-se o seguinte comando para apenas extrair os termos e descrições

```
.\pdftotext -f 24 -l 114 -raw "Glossário de Enfermagem.pdf"
```

Em resumo, os parâmetros utilizados permitiram obter ficheiros de texto limpos de artefactos visuais, focando apenas na sequência de caracteres que compõem o corpo do dicionário:

- **-f [número]:** Define a página inicial para evitar a extração de prefácios.
- **-l [número]:** Define a página final (opcional, usado para testes de extração).
- **-raw:** Mantém a sequência de leitura lógica em vez da disposição física.

4.2 Extração no Documento de Medicina

A extração de informação no documento de medicina baseou-se numa abordagem estruturada, tirando partido da existência de identificadores numéricos que delimitam cada entrada. Para tal, foi feito primeiro uma limpeza do documento em que se retirou a palavra "Vocabulário" que aparecia frequentemente com o número de página e de seguida foi utilizada uma expressão regular que permite segmentar o texto em blocos correspondentes a cada termo do dicionário.

A expressão regular utilizada foi a seguinte:

```
1 padrao_bloco = r'^(\d+)\s+(.*?)(?=\n\d+\s|\Z)'
```

Esta expressão pode ser decomposta em vários componentes, cada um com uma função específica no processo de extração:

- **^** – Indica o início de uma linha. Em conjunto com a flag `re.MULTILINE`, permite aplicar o padrão a cada linha do texto.
- **(\d+)** – Este grupo entre parênteses é um *capturing group*. Os parênteses indicam que o conteúdo correspondente deve ser extraído e armazenado separadamente. Neste caso, captura o identificador numérico de cada entrada, permitindo utilizá-lo posteriormente como ID.

- `\s+` – Representa um ou mais espaços em branco, funcionando como separador entre o identificador e o conteúdo.
- `(.*)` – Este é também um *capturing group*, responsável por capturar todo o conteúdo da entrada. O uso de `.*?` (modo não guloso) garante que a captura é limitada ao mínimo necessário, evitando que consuma texto pertencente a entradas seguintes.
- `(?=...)` – Este padrão representa uma *lookahead* positiva. Embora utilize parênteses, não corresponde a um grupo de captura. Em vez disso, define uma condição que deve ser satisfeita para que a correspondência termine, sem incluir esse conteúdo no resultado final.
- `\n\d+\s` – Dentro da lookahead, este padrão indica que a captura deve parar quando é encontrada uma nova linha seguida de outro identificador numérico, sinalizando o início de uma nova entrada.
- `|` – Operador lógico “ou”, permitindo que a condição de paragem seja uma de várias alternativas.
- `\Z` – Representa o fim do texto, garantindo que a última entrada também é capturada corretamente, mesmo não sendo seguida por outra.

É importante destacar o papel dos parênteses na expressão regular. Existem dois tipos principais neste contexto: os *capturing groups*, utilizados para extrair diretamente partes relevantes do texto (como o ID e o conteúdo), e as estruturas como a *lookahead*, que utilizam parênteses para definir condições lógicas sem capturar informação. Esta distinção é fundamental para garantir que apenas os dados pretendidos são armazenados, enquanto os delimitadores são usados apenas como referência.

A utilização desta expressão regular permite segmentar eficazmente o documento em blocos independentes, cada um contendo toda a informação relativa a um termo específico.

Após a segmentação dos blocos, foi utilizada uma segunda expressão regular para extrair as traduções associadas a cada língua. A expressão utilizada foi a seguinte:

```
1 regex_trads = r'^(es|en|pt|la)\s+(.*) (?:\n(?:es|en|pt|la)
2 )\s|\Z)'
```

Esta expressão permite identificar cada tradução com base na sigla da língua no início de cada linha. A sua decomposição é a seguinte:

- `^` – Indica o início de uma linha. Quando utilizado com a flag `re.MULTILINE`, este símbolo aplica-se ao início de cada linha do texto, permitindo encontrar traduções distribuídas ao longo do bloco.
- `(es|en|pt|la)` – Este é um *capturing group* que identifica as diferentes línguas possíveis. O operador `|` funciona como um “ou”, permitindo corresponder a qualquer uma das siglas. O facto de estar entre parênteses permite capturar diretamente a língua.
- `\s+` – Representa um ou mais espaços em branco, separando a sigla da língua do conteúdo da tradução.
- `(.*?)` – Este segundo *capturing group* captura o conteúdo da tradução. O uso do quantificador não guloso `.*?` garante que a captura é limitada ao conteúdo correspondente a essa língua.
- `(?=...)` – Define uma *lookahead* positiva, que indica onde a captura deve terminar, sem consumir esse conteúdo.
- `\n(?:es|en|pt|la)\s` – Dentro da *lookahead*, este padrão indica que a captura termina quando é encontrada uma nova linha seguida de outra sigla de língua. O uso de `(?:...)` define um grupo não capturante, utilizado apenas para agrupamento lógico sem armazenar o resultado.
- `|` – Operador lógico “ou”, permitindo considerar múltiplas condições de paragem.
- `\Z` – Representa o fim do texto, garantindo que a última tradução também é capturada corretamente.

Antes de aplicar esta expressão, é construída uma nova string contendo apenas as linhas correspondentes às traduções. Para tal, o conteúdo do bloco é dividido em linhas, sendo a primeira linha — que corresponde ao termo principal — removida. As restantes linhas, que contêm as traduções nas diferentes línguas, são posteriormente reunidas numa única string.

```
1
2 texto_traducoes = "\n".join(linhas[1:])
```

A extração é então realizada com:

```
1
2 trads_encontradas = re.findall(regex_trads,
    texto_traducoes, re.DOTALL | re.MULTILINE)
```

A função `re.findall` devolve uma lista de tuplos, onde cada tuplo corresponde a um *capturing group* definido na expressão regular. Neste caso, cada elemento da lista contém dois valores: a sigla da língua e o respetivo texto da tradução.

A utilização conjunta das flags `re.MULTILINE` e `re.DOTALL` é fundamental. A flag `re.MULTILINE` permite que o símbolo `^` corresponda ao início de cada linha, enquanto a flag `re.DOTALL` faz com que o metacaractere “.” capture também quebras de linha. Desta forma, é possível identificar corretamente cada tradução, mesmo quando o seu conteúdo se estende por várias linhas.

O resultado final é uma estrutura que pode ser facilmente convertida num dicionário, associando cada língua ao seu respetivo conteúdo.

4.3 Extração no Documento de COVID-19

No caso do documento WIPO Pearl COVID-19, a ausência de uma estrutura rígida e a presença de caracteres internacionais exigiram uma abordagem algorítmica determinística. O *parser* foi desenhado como um sistema “blindado”, utilizando as categorias científicas como âncoras para segmentar a informação.

O processo de extração seguiu a seguinte lógica estruturada:

- **Limpeza Cirúrgica e Filtragem de Ruído:** Inicialmente, o texto bruto é filtrado para remover artefactos de paginação e metadados irrelevantes que poderiam causar falsos positivos nas expressões regulares.
- **Identificação de Âncoras Estruturais:** O algoritmo identifica os índices das linhas que contêm as categorias (ex: MEDI, CHEM, entre outros...), que servem como o pilar de cada entrada. Tudo o que precede a categoria é metainformação em Inglês; o que a sucede são as traduções.

```
1 padrao_categoria = re.compile(r'^(MEDI|CHEM|ENVR|SCIE  
    ),\s')  
2 idx_cats = [i for i, l in enumerate(linhas) if  
    padrao_categoria.match(l)]
```

Lógica da Regex: O símbolo `^` é fundamental nesta expressão, pois obriga a que a correspondência ocorra estritamente no início da linha, prevenindo falsos positivos caso estas siglas surjam no meio de uma definição. O grupo lógico `(MEDI|CHEM|...)` define os prefixos aceites, enquanto `[\s]` assegura que a sigla é validada apenas se for imediatamente seguida de uma vírgula ou de um espaço em branco.

- **Segmentação de Termos, Sinónimos e Definições:** A lógica assume que a primeira linha do bloco é o termo principal. De seguida, verifica-se a existência de sinónimos através do marcador (syn.), isolando o restante texto como a definição.

```

1 if def_start < idx_cat and linhas[def_start].
    startswith('(syn.)'):
2     sinonimo = linhas[def_start].replace('(syn.)', ' ')
        .strip()
3     def_start += 1
4 definicao = "␣".join(linhas[def_start:idx_cat])

```

- **Motor de Tradução Multilíngue:** Para extrair as traduções, utilizou-se uma expressão regular que permite capturar a sigla do idioma mesmo quando o texto associado se encontra na linha seguinte, garantindo a integridade em alfabetos não latinos.

```

1 padrao_lingua = re.compile(r'^(AR|DE|ES|FR|JA|KO|PT|
    RU|ZH)(?:\s+(.*))?$')

```

Lógica da Regex: A robustez desta expressão reside na combinação de grupos de captura e condicionalidade. O primeiro grupo captura a sigla do idioma no início da linha. O segundo bloco, `(?:\s+(.*))?$`, é um grupo não-capturante tornado opcional pelo quantificador `?` no final. Isto significa que, se a linha contiver a sigla e o texto à frente, extrai ambos; mas se a linha contiver **apenas** a sigla (comum quando a formatação salta o texto árabe ou chinês para a linha de baixo), a expressão não falha, permitindo ao ciclo registar a mudança de idioma e capturar o texto na iteração seguinte.

- **Heurística de Paragem por Validação Unicode:** Como o Chinês (ZH) é o último idioma de cada entrada, o sistema utiliza uma validação de caracteres Unicode para detetar o fim do bloco. Se encontrar caracteres latinos num contexto de tradução chinesa, o *parser* interrompe a leitura e inicia o próximo conceito.

```

1 if current_lang == 'ZH':
2     if not re.search(r'[\u4e00-\u9fff]', 1) and re.
        search(r'[a-zA-Z]', 1):
3         break

```

Lógica da Regex: Em vez de procurar delimitadores de parágrafo estruturais (que falham frequentemente na conversão de PDF), a lógica

interroga diretamente a tabela *Unicode*. A expressão `[\u4e00-\u9fff]` procura ideogramas da família CJK (Chinês, Japonês e Coreano). Se a iteração atual, que deveria ser chinesa, não acusar a presença destes ideogramas e, simultaneamente, acusar a presença de caracteres do alfabeto latino (`[a-zA-Z]`), o algoritmo obtém a certeza matemática de que abandonou a secção de traduções e atingiu o termo principal (em inglês) do conceito seguinte.

Esta metodologia modular permitiu converter um documento altamente heterogêneo num formato estruturado, preservando a relação semântica entre os termos e as suas dez variações linguísticas.

4.4 Extração no Documento de Neologismos

A extração de informação no glossário de neologismos revelou-se um desafio particular devido à sua natureza menos formal e à ausência de delimitadores tipográficos claros (como IDs ou quebras de bloco consistentes). Neste caso, foi necessário adotar uma abordagem sequencial baseada em iteradores (*while loop*), utilizando a informação gramatical como a âncora principal para identificar novos conceitos.

O processo de *parsing* assentou nos seguintes mecanismos lógicos:

- **Tratamento de Codificação e Leitura Segura:** Sendo um documento em português com elevada presença de diacríticos e eventuais corrupções de conversão do PDF, o ficheiro foi aberto recorrendo à codificação `latin-1` com substituição de erros, garantindo que o algoritmo não falhasse perante caracteres inválidos.

```
1 with open(caminho_txt, "r", encoding="latin-1",
2           errors="replace") as f:
    linhas = f.readlines()
```

- **Identificação de Termos (A Âncora Gramatical):** Uma vez que os termos não possuíam numeração, o sistema foi programado para procurar as abreviaturas de género gramatical (`s.f.` para substantivo feminino e `s.m.` para substantivo masculino).

```
1 match_genero = re.search(r'\s+(s\.f\.|s\.m\.)', linha
2 )
3 if match_genero:
4     palavra = linha[:match_genero.start()].strip()
5     genero = match_genero.group(1).strip()
```

Lógica da Regex: A expressão regular `\s+(s\.f\.|s\.m\.)` atua como um separador sintático. O prefixo `\s+` exige obrigatoriamente a presença de um ou mais espaços em branco antes da sigla; isto é uma salvaguarda crítica contra falsos positivos, garantindo que o *parser* não corta acidentalmente palavras que, por mera coincidência linguística, terminem nos mesmos caracteres. O grupo de captura (...) emprega o operador lógico *OR* (|) para identificar a variação de género. Além disso, o uso da barra invertida (\.) atua como *escape*, forçando a *Regex* a interpretar o ponto como um sinal de pontuação literal, em vez do metacaractere *wildcard* (que faria correspondência com qualquer letra). Ao isolar a posição inicial desta âncora (`match_genero.start()`), o algoritmo recorta cirurgicamente tudo o que está à sua esquerda, garantindo a extração do termo puro.

- **Extração de Traduções Condicionais:** Após identificar o termo principal, o *parser* avança para a linha seguinte e verifica se esta contém as *tags* tradutivas [ing] (Inglês) ou [esp] (Espanhol). Caso existam, a linha é dividida pelo delimitador ponto e vírgula (;) e as *tags* são limpas. Se não existirem, o algoritmo assume que a definição já começou, não perdendo informação.
- **Captura da Definição e Heurística de Retrocesso (*Backtracking*):** Para extrair a definição, que frequentemente se estende por múltiplas linhas, utilizou-se um ciclo de leitura secundário. Este ciclo possui dois critérios de paragem cruciais: encontrar uma linha vazia ou detetar prematuramente o género gramatical do **próximo** termo. Se a segunda condição se verificar, o índice de leitura recua um passo (`i -= 1`), permitindo que o ciclo principal capture o novo conceito na iteração seguinte.

```
1 while i < total_linhas:
2     prox_linha = linhas[i].strip()
3     if prox_linha == "":
4         break
5     if re.search(r'\s+(s\.f\.|s\.m\.)', prox_linha):
6         i -= 1
7         break
8     definicao_linhas.append(prox_linha)
9     i += 1
```

Esta abordagem iterativa com controlo manual do índice provou ser altamente eficaz para documentos onde a formatação flutua e onde a estrutura

do bloco tem de ser inferida através de marcadores linguísticos, em vez de delimitadores de *layout*.

4.5 Extração no Documento do Ministério da Saúde

O documento proveniente do Ministério da Saúde apresentou um formato particular onde o pilar central de cada conceito não é o termo em si, mas sim o campo “Categoria:”, que surge invariavelmente intercalado entre o termo e a sua definição. Para processar esta estrutura, desenvolveu-se um algoritmo baseado em âncoras centrais e pesquisa bidirecional (retrocesso para o termo e avanço para a definição).

A implementação do *parser* seguiu as seguintes etapas lógicas:

- **Leitura Segura e Limpeza Estrutural:** Sendo um documento rico em vocabulário técnico em português, a leitura foi forçada a `latin-1` para preservar diacríticos. De seguida, aplicou-se uma limpeza intensiva para remover números de página, cabeçalhos residuais alfanuméricos e referências cruzadas (ex: “Ver Abuso sexual.”), que fragmentariam a continuidade do texto.

```
1 for linha in linhas_brutas:
2     l = linha.strip()
3     if not l or l.isdigit(): continue
4     if len(l) == 1 and l.isalpha(): continue
5     if re.match(r'^\d+\s+[A-Z]+$', l): continue
6     if l.startswith("Ver ") and l.endswith("."):
7         continue
8     linhas_limpas.append(l)
```

Lógica da Regex: A expressão regular `^\d+\s+[A-Z]+$` atua como um filtro estrito para cabeçalhos residuais ou marcadores de índice soltos. A utilização simultânea das âncoras de início (^) e de fim de linha (\$) é o detalhe técnico mais importante desta expressão: ela garante que o padrão só faz correspondência se a linha **inteira** for composta exclusivamente por esse formato. Isto protege definições legítimas que, por coincidência, pudessem conter este padrão a meio de uma frase. O núcleo da expressão exige a presença de um ou mais algarismos (`\d+`), seguidos de pelo menos um espaço (`\s+`) e terminando obrigatoriamente numa sequência de uma ou mais letras maiúsculas (`[A-Z]+`). O uso deste padrão permite varrer de forma cirúrgica lixo de formatação (e.g., “16 ACI”) resultante da quebra de colunas do PDF.

- **Identificação das Âncoras Centrais:** Em vez de tentar identificar o início de cada termo diretamente (o que seria propenso a erros devido à formatação variável), o algoritmo mapeia o índice exato de todas as linhas que começam pela *string* “Categoria:”, que servem de marco zero para cada conceito.

```
1 indices_categorias = [  
2     i for i, linha in enumerate(linhas_limpas)  
3     if linha.startswith("Categoria:")  
4 ]
```

- **Extração do Termo por Retrocesso:** Sabendo onde está a categoria, o algoritmo itera para trás ($j -= 1$) para capturar as linhas que compõem o termo. O critério de paragem fundamental deste retrocesso é encontrar uma linha que termine em ponto final (.), o que indica inequivocamente que se atingiu o final da definição do termo anterior.

```
1 linhas_palavra = []  
2 j = idx_cat - 1  
3 while j >= 0:  
4     if j < idx_cat - 1 and linhas_limpas[j].endswith(  
5         '.'):   
6         break  
7     linhas_palavra.insert(0, linhas_limpas[j])  
8     j -= 1  
9 inicio_palavra = j + 1  
10 palavra = "␣".join(linhas_palavra)
```

- **Delimitação e Extração da Definição:** Após o mapeamento de todos os termos e categorias, a extração das definições torna-se um simples processo de fatiamento de listas (*list slicing*). A definição de um conceito começa na linha seguinte à sua categoria e termina exatamente na linha onde começa o termo do conceito seguinte.

```

1 for i, bloco in enumerate(blocos):
2     linha_inicio_def = bloco["idx_cat"] + 1
3
4     if i + 1 < len(blocos):
5         linha_fim_def = blocos[i+1]["inicio_palavra"]
6     else:
7         linha_fim_def = len(linhas_limpas)
8
9     texto_definicao = "\n".join(
10         linhas_limpas[linha_inicio_def:linha_fim_def]
11     ).strip()

```

Esta arquitetura bidirecional garantiu uma precisão de extração elevadíssima, transformando um documento visualmente orientado num formato hierárquico perfeitamente indexado no dicionário global.

4.6 Extração no Documento de Enfermagem

Antes da aplicação das heurísticas de extração, foi realizada uma etapa de limpeza do documento com o objetivo de remover ruído estrutural e garantir consistência no texto.

Nesta fase, foi inicialmente removido o cabeçalho recorrente do documento, que se estendia desde o título do glossário até à secção de contexto, através da seguinte expressão regular:

```

1 header_pattern = r'GLOSS RIO DA LINGUAGEM ESPECIAL DE
2   ENFERMAGEM.*?CONTEXTO AMAZÔNICO'
3 texto = re.sub(header_pattern, '', texto, flags=re.DOTALL)

```

Esta expressão utiliza o metacaractere `.*?` de forma não gulosa, permitindo remover todo o conteúdo entre o início do cabeçalho e a marca final "CONTEXTO AMAZÔNICO". A utilização da flag `re.DOTALL` é essencial neste contexto, uma vez que permite que o ponto (`.`) corresponda também a quebras de linha, garantindo a remoção correta de blocos multi-linha.

De seguida, procedeu-se à remoção de números de página isolados, frequentemente presentes no documento e que interferiam com o processo de segmentação:


```
1 texto = re.sub(r'\n\s*\d+\s*\n', '\n\n', texto)
```

Esta expressão identifica linhas contendo apenas números (com possíveis espaços em branco) e substitui-as por quebras de linha normalizadas, contribuindo para uma estrutura textual mais uniforme e adequada à extração sequencial.

Esta etapa de limpeza foi essencial para reduzir ruído estrutural e melhorar a fiabilidade das heurísticas utilizadas na identificação de termos e definições.

Após a normalização do texto, foi utilizada uma expressão regular para segmentar o conteúdo em blocos correspondentes a cada entrada:

```
1 padrao_bloco = r'(?:^(\n)([A-Z - ][\n]+)\n(?:\s*FONTE
:)(.+?)\n\s*FONTE:\s*\n?(.*?)(?=\n\s*[A-Z - ][\n]+\n
n(?:\s*FONTE:)|\n\n|\Z)'
```

Esta expressão pode ser decomposta em vários componentes, cada um com uma função específica no processo de extração:

- `([A-ZÀ-Ú][^a-z\n\d]{2,})` – Este primeiro *capturing group* identifica o termo. Procura uma letra maiúscula inicial seguida de pelo menos dois caracteres que não sejam minúsculas, dígitos ou quebras de linha, isolando assim os títulos das entradas.
- `\n+` – Representa uma ou mais quebras de linha que separam o termo do início da sua definição.
- `(.+?)` – Este segundo *capturing group* captura o conteúdo da definição. O uso do quantificador não guloso garante que a captura pare assim que o marcador de fonte seja encontrado.
- `FONTE:\s*\n?(.*?)` – Identifica o literal “FONTE:” e utiliza um terceiro *capturing group* para extrair a referência bibliográfica associada, permitindo isolar este metadado do campo da definição.
- `(?=...)` – Representa uma *lookahead* positiva que define a condição de paragem da extração de um bloco sem consumir o texto seguinte.
- `[A-ZÀ-Ú][^a-z\n\d]{2,}\n` – Dentro da *lookahead*, este padrão indica que a captura deve terminar quando é detetado o início de um novo termo (maiúsculas seguidas de quebra de linha).
- `\Z` – Representa o fim do texto, garantindo que o último termo do glossário é capturado corretamente.

Tal como no documento anterior, a distinção entre grupos de captura e *lookahead* é fundamental para garantir que apenas os dados pretendidos são armazenados, enquanto os delimitadores estruturais (como o marcador “FONTE:”) são usados apenas como referência para a segmentação.

Finalmente, foi aplicada uma etapa de pós-processamento para normalizar o espaçamento das definições e filtrar falsos positivos:

```
1 definicao_limpa = re.sub(r'\s+', ' ', definicao)
2 if len(termo.split()) < 10:
3     res[termo] = definicao_limpa
```

O uso de `re.sub` permite colapsar múltiplas quebras de linha e espaços resultantes da extração num único espaço, garantindo a legibilidade do texto. A validação final assegura que apenas sequências curtas de palavras sejam consideradas termos, descartando eventuais parágrafos de ruído que tenham sobrevivido à limpeza inicial.

5 Convergência dos Dados

Após a extração individual de cada glossário, a etapa final do processamento consistiu na normalização e consolidação da informação num único recurso estruturado. Este processo, realizado pelo módulo `merger_json2.py`, é essencial para garantir que aplicações futuras possam consultar todos os termos de forma uniforme, independentemente da sua fonte original.

O pipeline de convergência seguiu uma lógica de mapeamento sistemático:

- **Carregamento Seguro de Fontes:** Para evitar que a ausência de um ficheiro interrompesse o processo global, implementou-se uma função auxiliar de carregamento com tratamento de exceções.

```
1 def carregar_json(caminho):
2     try:
3         with open(caminho, 'r', encoding='utf-8') as
4             f:
5                 return json.load(f)
6     except FileNotFoundError:
7         print(f"Aviso: Ficheiro {caminho} não encontrado.")
8     return []
```

- **Normalização de Esquemas (Mapping):** Uma vez que cada *parser* utilizou chaves diferentes (e.g., `termo_en`, `palavra`, `termo`), criou-

se uma função de mapeamento para traduzir estas variações para um modelo de dados global comum.

```
1 def adicionar_ao_glossario(lista_origem, fonte,
2   chave_termo):
3     for item in lista_origem:
4         glossario_global.append({
5             "id_global": str(id_contador),
6             "termo": item.get(chave_termo, "n.d."),
7             "definicao": item.get("definicao", "n.d."),
8             "info_gramatical": item.get("
9               info_gramatical", "n.d."),
10            "categoria": item.get("categoria", "n.d."),
11            "traducoes": item.get("traducoes", {}),
12            "fonte": fonte
13        })
```

- **Tratamento de Estruturas Não-Lista:** O glossário de enfermagem, ao contrário dos outros que geraram listas de objetos, resultou num dicionário simples de pares chave-valor. O *merger* foi dotado de uma lógica específica para iterar sobre este formato e integrá-lo no esquema global.

```
1 for termo, definicao in dados_enfermagem.items():
2     glossario_global.append({
3         "id_global": str(id_contador),
4         "termo": termo,
5         "definicao": definicao,
6         "fonte": "Glossário de Enfermagem"
7         # ... restantes campos como n.d.
8     })
```

5.1 Dicionário Global Consolidado

O resultado final deste processo é o ficheiro `dicionario_global.json`, que serve como a base de conhecimento centralizada do projeto. Abaixo apresenta-se a estrutura de um objeto típico neste dicionário, demonstrando a uniformidade alcançada:

```

1  [
2    {
3      "id_global": "1",
4      "termo": "acute respiratory distress syndrome",
5      "definicao": "Severe form of acute lung injury...",
6      "info_gramatical": "n.d.",
7      "categoria": "MEDI, Pathology",
8      "traducoes": {
9        "PT": "s ndrome de dificuldade respirat ria aguda",
10       },
11       "ES": "s ndrome de dificultad respiratoria aguda",
12       "FR": "syndrome de d tresse respiratoire aigu "
13     },
14     "fonte": "WIPO Pearl COVID-19"
15   ]

```

Esta estrutura unificada permite que o sistema escale facilmente, bastando adicionar novos *parsers* e mapeá-los para este esquema para expandir a base terminológica.

6 *main2.py*

O ficheiro *main2.py* corresponde ao ponto de entrada principal do sistema desenvolvido, sendo responsável pela orquestração global do pipeline de Processamento de Linguagem Natural (PLN). Este script não realiza processamento direto dos dados, funcionando apenas como coordenador da execução dos diferentes módulos e garantindo a correta sequência de operações necessárias à geração do resultado final.

Quando executado, o programa desencadeia um fluxo de execução sequencial composto por três fases principais: extração de informação através dos parsers, geração de ficheiros intermédios em formato JSON e, por fim, convergência de todos os resultados num único ficheiro final.

Fluxo de Execução do Sistema

O pipeline é executado segundo uma ordem lógica de dependências, assegurando que todos os dados são processados antes da etapa de integração final. Assim, o script segue a seguinte sequência de execução:

1. Execução dos módulos de extração (parsers) para todos os documentos;

2. Geração dos ficheiros JSON intermédios correspondentes a cada parser;
3. Convergência dos dados através do módulo de agregação;
4. Produção do ficheiro JSON final unificado.

A execução dos parsers é realizada de forma sequencial, sendo cada módulo responsável pela leitura de um documento específico e pela criação da sua representação estruturada em JSON. Embora os documentos possam apresentar diferentes níveis de complexidade e estrutura, todos os parsers produzem um output intermédio com o mesmo propósito: normalizar a informação extraída.

Após a conclusão da execução de todos os parsers, é então invocado o módulo responsável pela integração dos dados, garantindo que todos os ficheiros JSON intermédios são combinados num único dicionário global.

Ponto de Entrada do Sistema

O bloco final do script:

```
1 if __name__ == "__main__":  
2     main()
```

assegura que o pipeline apenas é executado quando o ficheiro é chamado diretamente, evitando a execução automática em caso de importação como módulo. Este mecanismo garante a modularidade do sistema e permite a reutilização dos componentes individuais em outros contextos..

7 Conclusão

O presente trabalho prático atingiu com sucesso o objetivo de extrair, normalizar e unificar informação terminológica a partir de fontes bibliográficas acentuadamente heterogéneas na área da saúde. A conversão de múltiplos ficheiros PDF, desprovidos de estruturação computacional, num dicionário global em formato JSON demonstra a importância e a aplicabilidade prática do Processamento de Linguagem Natural (PLN) na organização de dados.

A adoção de uma arquitetura de software modular, suportada por *parsers* independentes e um gestor central (`main2.py`), revelou-se uma decisão metodológica importante, sendo que, esta separação de responsabilidades não só facilitou o isolamento e correção de erros, como tornou o sistema escalável para a futura inclusão de novos glossários.

Do ponto de vista algorítmico, o projeto comprovou a versatilidade das Expressões Regulares (Regex) quando aliadas a heurísticas de programação dinâmicas. Ao longo do trabalho, ficou demonstrado que não existe uma solução única para o *parsing* de documentos. Foi necessário empregar desde estratégias de pesquisa espacial e validação de caracteres *Unicode* (no caso do dicionário multilingue da WIPO), até lógicas iterativas com retrocesso (*backtracking*) e ancoragem gramatical (nos dicionários de Neologismos e do Ministério da Saúde).

Apesar dos excelentes resultados obtidos, o desenvolvimento evidenciou algumas das limitações clássicas dos sistemas de extração baseados em regras. A fiabilidade destes algoritmos é altamente dependente da qualidade da extração inicial do texto (*pdftotext*) e de uma formatação tipográfica minimamente expectável, o que reforça o papel crítico que a fase de limpeza e normalização inicial desempenhou no sucesso da extração.

Em suma, o dicionário estruturado e consolidado resultante deste projeto constitui um recurso computacional e linguístico de elevado valor. Fica assim estabelecida uma base de conhecimento sólida, pronta a ser integrada em futuras aplicações de Engenharia Biomédica, tais como sistemas de recuperação de informação clínica, motores de pesquisa especializados ou ferramentas de auxílio ao diagnóstico e tradução médica.